DS1: Corrigé

4 octobre 2025

Exercices

Exercice 1. Fait en cours / TD.

Exercice 2.

Q1. Il y avait plusieurs possibilités à chaque fois.

a) `A || B`

b) `(A || C) && !(A && C)` ou encore `(A && !C) || (!A && C)`

c) `!(A || B ||C)` ou encore `!A && !B && !C`

d) L'implication $x \Rightarrow y$ peut se traduire par "(non x) ou y", d'où : `(!A || !C) && (C || A)`. En fait, c'est la même formule que la b) !

e) On veut avoir exactement 0 ou 2 des 3 variables qui sont vraies. On peut énumérer les cas :

```
(!A && !B && !C) || (A && B && !C) || (A && !B && C) || (!A && B && C)
// aucune vraie      A et B vraies      A et C vraies      B et C vraies
```

Q2. Il y a 3 points à corriger :

— Le deuxième format devrait être `%f` pour lire un flottant

— Il ne faut pas de `\n` dans scanf

— Il manque les `&` avant les variables

Solution :

```
// lit un entier et un flottant et les stocke dans x et y
scanf("%d %f", &x, &y);
```

Q3. L'initialisation du générateur aléatoire, avec `srand`, ne doit pas se faire dans la fonction, mais dans le main. En effet, si l'on met l'instruction `srand(time(NULL));` au début de la fonction `pile_face`, chaque appel à la fonction réinitialise le générateur ! En particulier, si on appelle la fonction deux fois dans le main, il y a de grandes chances qu'elle s'exécute au sein d'une seule seconde, et qu'elle renvoie donc exactement le même résultat !!

```
int pile_face(){
   int nb_lancers = 0;
   bool pile_obtenu = false;
   while (!pile_obtenu){
      pile_obtenu = (rand()%2 == 0); // une chance sur 2
      nb_lancers += 1;
   }
   return nb_lancers;
}
```

Q4.

```
int main(){
   srand(time(NULL));
   int nb_faces = pile_face();
   printf("%d faces\n", nb_faces);
}
```

Exercice 3.

Q1. *Coquille dans le sujet : c'était "plus qu'une pièce de 5" et pas deux.*
Soit n l'entier à décomposer, et x_1, x_2, x_5, x_{10} le nombre de pièces de 1, 2, 5 et 10 utilisées dans la décomposition.
Supposons que $x_1 > 1$. Alors on peut exhiber une meilleure solution, en remplaçant deux pièces de 1 par une de 2. Autrement dit, $(x_1 - 2, x_2 + 1, x_5, x_{10})$ est une solution utilisant une pièce de moins que (x_1, x_2, x_5, x_{10}).
De même, si $x_2 > 2$, alors la solution $(x_1 + 1, x_2 - 3, x_5 + 1, x_{10})$ utilise une pièce de moins.
Enfin, si $x_5 > 1$, $(x_1, x_2, x_5 - 2, x_{10} + 1)$ est une meilleure solution.

Q2. L'idée est d'enlever le plus de pièces de 10, puis d'enlever le plus de pièces de 5, puis de 2, puis de 1, ce qui revient à faire des divisions entières successives :

```c
/*
Décompose n>=0 centimes en utilisant des pièces de 1, 2, 5, 10 centimes,
en utilisant le moins de pièces possibles, et affiche la décomposition.
Exemple: pour decomp(38), la fonction affichera:
  3 pièce(s) de 10 centimes,  1 pièce(s) de 5 centimes,
  1 pièce(s) de 2 centimes,1 pièce(s) de 1 centime */
void decomp(int n){
   int x_10 = n/10;
   n = n%10;
   int x_5 = n/5;
   n = n%5;
   int x_2 = n/2;
   n = n%2;
   int x_1 = n;
   printf("%d pièce(s) de 10 centimes", x_10);
   printf("%d pièce(s) de  5 centimes", x_5);
   printf("%d pièce(s) de  2 centimes", x_2);
   printf("%d pièce(s) de  1 centime",  x_1);
}
```

On pouvait passer par une fonction auxiliaire qui réduit un peu la quantité de code à écrire sur papier :

```c
int retirer(int n, int p){
   int nb_pieces = n/p;
   printf("%d pièce(s) de %d centimes", nb_pieces, p);
   return n % p;
}

void decomp(int n){
   n = retirer(n, 10);
   n = retirer(n, 5 );
   n = retirer(n, 2 );
   n = retirer(n, 1 );
}
```

Exercice 4.

Q1. Tableau des valeurs :

Nb de tours	0	1	2	3
k	0	1	2	3
s	1	4	9	16

Q2.

```
/* Renvoie le plus grand entier k tel que k^2 <= n.
   Hypothèse: n >= 0 */
int racine(int n){
   assert(n >= 0);
   ...
}
```

Q3. Montrons par récurrence que pour tout $i \in \mathbb{N}$, $P(i)$: "$s_i = (k_i + 1)^2$".

— Initialisation ($i = 0$) : $k_0 = 0$ et $s_0 = 1 = (0 + 1)^2$, d'où $P(0)$.

— Hérédité : Soit $i \in \mathbb{N}$ tel que $P(i)$ et tel que la boucle fait $i + 1$ passages. Alors, $k_{i+1} = k_i + 1$ et $s_{i+1} = s_i + 2k_{i+1} + 1$ (*Il fallait faire attention au fait que k est modifié AVANT s, et donc que c'est la nouvelle valeur de k qui intervient pour le calcul de s*). Ainsi :

$$\begin{array}{rcll} s_{i+1} & = & s_i + 2k_{i+1} + 1 & \\ & = & (k_i + 1)^2 + 2k_{i+1} + 1 & \text{par HR} \\ & = & k_{i+1}^2 + 2k_{i+1} + 1 & \\ & = (k_{i+1} + 1)^2 & & \end{array}$$

D'où $P(i + 1)$.

Donc $s = (k + 1)^2$ est bien un invariant.

Q4. On propose l'invariant : $k^2 \leq n$.
Montrons donc par récurrence que $\forall i \in \mathbb{N}$, $P(i)$: "$k_i^2 \leq n$".

— Initialisation ($i = 0$) : $k_i = 0$ et $n \in \mathbb{N}$, d'où $P(0)$.

— Hérédité : Soit $i \in \mathbb{N}$ tel que $P(i)$ et tel que la boucle fait $i + 1$ passages. Alors, $k_{i+1} = k_i + 1$, et puisque l'on fait un passage de plus, la condition de boucle est respectée donc $s_i \leq n$. Or, d'après l'invariant montré en Q3, $s_i = (k_i + 1)^2 = k_{i+1}^2$. Donc $k_{i+1}^2 \leq n$, d'où $P(i + 1)$.

En sortie de boucle, on a $s > n$ par condition de boucle, et donc, en utilisant les deux invariants, on a bien $k^2 \leq n$ et $n < s = (k+1)^2$. k est bien la racine entière de n en sortie de boucle, et l'algorithme renvoie bien k, d'où la correction.

Q5. On considère $V = n - k^2$. C'est une quantité entière et strictement décroissante car n et k sont des `int` et k augmente de 1 à chaque tour. De plus, nous avons montré que $k^2 \leq n$ est un invariant de boucle. Donc V reste positif : c'est bien un variant de boucle.

Q6.

```
int racine_cubique(int n)
    int k = 0;
    int s = 1; // Invariant: s = (k+1)^2
    int c = 1; // Invariant: c = (k+1)^3
    while (c <= n){
        k = k + 1;
        c = c + 3*s + 3*k + 1;
        s = s + 2*k + 1;
    }
    // en sortie, k^3 <= n < c = (k+1)^3
    return k;
```

Achat et vente d'actions

Q1. La phrase proposée n'est pas vraie, car le maximum peut être atteint avant le minimum alors que l'on doit acheter avant de vendre. Par exemple, pour une suite de deux jours X_0, X_1 avec $X_0 = 500$ et $X_1 = 0$, on ne peut pas acheter au jour 1 et vendre au jour 0.

Q2. Étant donné une date i_a d'achat, la meilleure date de vente correspondante est la date de valeur maximale **après** i_a, c'est à dire la date i_v telle que $X_{i_v} = \max\{X_j \mid j \in [\![i_a, n-1]\!]\}$.

Q3. Il manque une précondition à l'algorithme : il faut avoir $i_1 \leq i_2$ (sinon aucune date n'est possible). Il n'y avait pas besoin de montrer la correction, notons tout de même l'invariant expliquant le principe :

Algorithme 1 : `jour_max`$(n, X_0, \ldots, X_{n-1}, i_1, i_2)$

Entrée(s) : $n \in \mathbb{N}, X_0, \ldots, X_{n-1} \in \mathbb{R}^+, i_1, i_2 \in [\![0, n-1]\!]$
Sortie(s) : $i_m \in [\![i_1, i_2]\!]$ tel que X_{i_m} est maximale

```
i_m ← i_1;
i ← i_1 + 1;
// Invariant: X_{i_m} est le max de X_{i_1} ... X_{i-1}
. tant que i ≤ i_2 faire
    si X_i > X_{i_m} alors
        i_m ← i;
    i ← i + 1;
// En sortie, i = i_2 + 1, donc X_{i_m} est bien le max cherché
retourner i_m
```

Q4. L'algorithme `profit` teste toutes les dates d'achat possible, en partant de la dernière. Pour chaque date d'achat, il calcule la meilleure date de vente, et il garde en mémoire le meilleure couple (date d'achat, date de vente) vu jusqu'à présent.
Rôle des variables :

- i : pour parcourir toutes les dates d'achat potentielles ;
- i_a : meilleure date d'achat vue pour l'instant (i.e. entre $i+1$ et $n-1$) ;
- i_v : meilleure date de vente pour i_a ;
- j : meilleure date de vente pour i à chaque tour.

Q5. La boucle tant que peut se réécrire "Pour i allant de $n-1$ à 0 par pas de (-1)". Elle termine donc forcément. Alternativement, on remarque que $i+1$ est un variant de boucle.

Q6. En résumant les rôles des variables, on peut considérer la propriété I suivante :

$$X_{i_v} - X_{i_a} = \max\{X_t - X_s \mid s, t \in [\![i+1, n-1]\!], s \leq t\}$$

Autrement dit, i_v et i_a sont bien des dates de profits maximal parmi les dates vues pour l'instant.

Posons, pour $k \in \mathbb{N}$, i_k la valeur de i après k tours de boucle, et idem pour $i_{a,k}$, $i_{v,k}$. Montrons par récurrence que $\forall k \in \mathbb{N}, I(k)$, où $I(k)$ est la propriété I où chaque variable i, i_a, i_v a été indexée par k.

- Initialisation ($k = 0$) : $i_{a,0} = i_{v,0} = n-1$ et $i_0 = n-2$. Il faut vérifier que $X_{n-1} - X_{n-1} = \max\{X_{n-1} - X_{n-1}\}$: c'est trivialement vrai. D'où $I(0)$.
- Hérédité : Soit $k \in \mathbb{N}$ tel que $I(k)$ et tel que l'algorithme fait $k+1$ tours. Notons $j =$ `jour_max`$(n, X_0, \ldots, X_{n-1}, i_k, n-1)$. Alors, par propriété de `jour_max`, $X_j - X_{i_k}$

est le maximum de $\{X_{i_k} - X_{i_k}, X_{i_k+1} - X_{i_k}, \dots, X_{n-1} - X_{i_k}\}$.
Distinguons deux cas :

— Si $X_j - X_{i_k} > X_{i_{v,k}} - X_{i_{a,k}}$:
Alors $i_{a,k+1} = i_k$, $i_{v,k+1} = j$, et $i_{k+1} = i_k - 1$. Notons qu'alors, $X_{i_{v,k+1}} - X_{i_{v,k+1}} > X_{i_{v,k}} - X_{i_{v,k}}$ Il reste donc à vérifier que pour tout $s \leq t \in [\![i_k, n-1]\!]$, $X_{i_{v,k+1}} - X_{i_{v,k+1}} > X_t - X_s$. Soient $s \leq t \in [\![i_k, n-1]\!]$:

  — Si $s \geq i_k + 1$, alors par HR, $X_{i_{v,k}} - X_{i_{v,k}} > X_t - X_s$, et on a bien $X_{i_{v,k+1}} - X_{i_{v,k+1}} > X_t - X_s$.

  — Sinon, alors $s = i_k$, et par propriété de `jour_max`, puisque $i_{a,k+1} = i_k$ et $i_{v,k+1} = \texttt{jour_max}(i_k)$, on a bien $X_{i_{v,k+1}} - X_{i_{v,k+1}} > X_t - X_s$.

— Si $X_j - X_{i_k} \leq X_{i_{v,k}} - X_{i_{a,k}}$: Alors $i_{a,k+1} = i_{a,k}$, $i_{v,k+1} = i_{v,k}$, et $i_{k+1} = i_k - 1$. Notons qu'à nouveau, $X_{i_{v,k+1}} - X_{i_{v,k+1}} > X_{i_{v,k}} - X_{i_{v,k}}$. Un raisonnement analogue au premier cas montre que $X_{i_{v,k+1}} - X_{i_{v,k+1}}$ est plus grand que tout $X_t - X_s$ avec $s \leq t \in [\![i_k, n-1]\!]$.

Finalement, $I(k+1)$ est bien vérifiée dans tous les cas.

Ainsi, I est bien un invariant de boucle. En particulier, en sortie de boucle, on a $i = -1$, et I dit précisément que i_a, i_v sont des indices de profit maximal.

Q7. On se rend compte que la variable j sert à stocker l'indice du max entre i et $n-1$. Plutôt que de le recalculer à chaque tour, on peut le mettre à jour, en comparant X_i et X_j. Ceci permet d'éliminer l'appel à `jour_max` :

Algorithme 2 : `profit2`$(n, X_0, \dots, X_{n-1})$

Entrée(s) : $n \in \mathbb{N}, X_0, \dots, X_{n-1} \in \mathbb{R}^+$
Sortie(s) : $i_a, i_v \in [\![0, n-1]\!]$ indices de profit maximal

```
i_a ← n − 1;
i_v ← n − 1;
i ← n − 2;
j ← n − 1;
// Invariant: j = jour_max(n, X_0, ..., X_{n−1}, i + 1, n − 1)
tant que i ≥ 0 faire
    si X_i > X_j alors
        j ← i;
    si X_j − X_i > X_{i_v} − X_{i_a} alors
        i_a ← i;
        i_v ← j;
    i ← i − 1;
```

Algorithme de Bresenham

Q1. Il faut que $x_A < x_B$ et $\frac{y_B - y_A}{x_B - x_A} \in [0, 1]$.

Q2. Pour $(A, B) \in O_1$ et $x \in \mathbb{R}$, $Y_{AB}(x) = \frac{y_B - y_A}{x_B - x_A}(x - x_A) + y_A$

Q3. La suite des valeurs des variables est :

Tour	0	1	2	3	4	5	6
x	0	1	2	3	4	5	6
y	0	1	2	2	3	4	5
e	0	-2	-4	4	2	0	-2

L'algorithme colorie donc les pixels de coordonnées $(0, 0), (1, 1), (2, 2), (3, 2), (4, 3)$ et $(5, 4)$.

Q4. Dans l'algorithme, à chaque tour de boucle, quand y ne change pas, l'écart $Y_{AB}(x) - y$ augmente de $\frac{d_y}{d_x}$ puisque x augmente de 1, et e augmente de $2d_y$. On peut donc supposer que $\gamma = 2d_x$, on aura alors $\frac{e}{\gamma}$ qui augmente à la même vitesse que $Y_{AB} - y$.

On introduit, pour $k \in \mathbb{N}$, x_k, y_k, e_k les valeurs de x, y, e après k tours de boucles. Montrons par récurrence $\forall k \in \mathbb{N}, P(k)$: "$\frac{e_k}{2d_x} = Y_{AB}(x_k) - y_k$".

- En entrée de boucle, $x_0 = x_A, y_0 = y_A$, donc $Y_{AB}(x_0) - y_0 = 0 = \frac{e_0}{2d_x}$.
- Soit $k \in \mathbb{N}$ tel que $P(k)$ et tel que l'on fait $k + 1$ passages. Alors :
  - Si $e_k + 2d_y \leq d_x$:
    $y_{k+1} = y_k, x_{k+1} = x_k + 1$ et $e_{k+1} = e_k + 2d_y$. On a :

$$\begin{array}{rcll} Y_{AB}(x_{k+1}) - y_{k+1} & = & Y_{AB}(x_k) + \frac{d_y}{d_x} - y_k & \\ & = & \frac{e_k}{2d_x} + \frac{d_y}{d_x} & \text{par HR} \\ & = & \frac{e_k + 2d_y}{2d_x} & \\ & = & \frac{e_{k+1}}{2d_x} & \end{array}$$

  - Sinon, $y_{k+1} = y_k + 1$, $x_{k+1} = x_k + 1$ et $e_{k+1} = e_k + 2d_y - 2d_x$. On a :

$$\begin{array}{rcll} Y_{AB}(x_{k+1}) - y_{k+1} & = & Y_{AB}(x_k) + \frac{d_y}{d_x} - y_k - 1 & \\ & = & \frac{e_k}{2d_x} + \frac{d_y}{d_x} - 1 & \text{par HR} \\ & = & \frac{e_k + 2d_y - 2d_x}{2d_x} & \\ & = & \frac{e_{k+1}}{2d_x} & \end{array}$$

  Finalement, $P(k + 1)$ est satisfaite.

D'où l'invariant annoncé.

Q5. On montre aussi l'invariant : $\frac{e}{2d_x} \in [-\frac{1}{2}, \frac{1}{2}]$. Ceci montrera bien qu'à chaque tour de boucle, l'écart entre $Y_{AB}(x)$ et y est inférieur à $\frac{1}{2}$, et donc que l'algorithme de Bresenham affiche bien les même pixels que l'algorithme naïf.

Montrons par récurrence que $\forall k \in \mathbb{N}$, $P(k)$: "$-d_x \leq e_k \leq d_x$" :

- Initialisation : $e_0 = 0$ et $d_x \geq 0$, donc $P(0)$ est vraie.
- Hérédité : Soit $k \in \mathbb{N}$ tel que $P(k)$, supposons que l'on fait un $k + 1$-ème passage de boucle.

- Si $e_k + 2d_y > d_x$, alors $e_{k+1} = e_k + 2d_y - 2d_x > -d_x$. De plus, puisque l'on trace une ligne de pente au plus 1, on a $d_x \geq d_y$, donc $e_{k+1} \leq e_k \leq d_x$ (par HR). On a bien $-d_x \leq e_{k+1} \leq d_x$, d'où $P(k+1)$.
- Sinon (c'est à dire si $e_k + 2d_y \leq d_x$), $e_{k+1} = e_k + 2d_y \leq d_x$ et $e_{k+1} \geq e_k \geq -d_x$ par HR. $P(k+1)$ est vérifiée.

$P(k+1)$ est vraie dans les deux cas, d'où l'hérédité.

D'où la correction de l'algorithme de Bresenham. *Notons qu'ici, la correction de l'algorithme ne s'exprime pas à la fin de la boucle, mais tout au long de l'exécution !*